

Lab12 – Troubleshooting Pod

In this exercise, you will practice your troubleshooting skills by inspecting a misconfigured Pod.

1. Create a new Pod from the YAML manifest in the file `pod.yaml` (./pod.yaml). Check the Pod's status. Do you see any issue?

```
vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f pod.yaml
pod/date-recorder created
vagrant@kubernetes-control-plane:~/lab$ kubectl get pod
NAME          READY   STATUS    RESTARTS   AGE
date-recorder  1/1     Running   0           61s
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```

2. Render the logs of the running container and identify an issue.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl logs date-recorder
[Error: ENOENT: no such file or directory, open '/root/tmp/startup-marker.txt'] {
  errno: -2,
  code: 'ENOENT',
  syscall: 'open',
  path: '/root/tmp/startup-marker.txt'
}
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```

3. Shell into the container. Can you verify the issue based on the rendered log message?

```
vagrant@kubernetes-control-plane:~/lab$ kubectl exec -it date-recorder -- /bin/sh
error: Internal error occurred: error executing command in container: failed to exec in container: failed to start exec "55c31f71769b4e82b9bed32bc52361108ddb2bda8e8f5ab2092ccabd2be9811d": OCI runtime exec failed: exec failed: unable to start container process: exec: "/bin/sh": stat /bin/sh: no such file or directory: unknown
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```

```
vagrant@kubernetes-control-plane:~/lab$ kubectl debug -it date-recorder --image=busybox --target=debian --share-processes
Targeting container "debian". If you don't see processes from this container it may be because the container runtime doesn't support this feature.
Defaulting debug container name to debugger-mzxfl.
If you don't see a command prompt, try pressing enter.
/#
/# ps
PID    USER      TIME  COMMAND
1      root      0:00  /nodejs/bin/node -e const fs = require('fs'); let timestamp = Date.now(); fs.writeFile('/root/tmp/startup-marker.txt', t
22     root      0:00  sh
27     root      0:00  ps
/#
```

Apparently, the directory we want to write to does indeed not exist.

4. Suggest solutions that can fix the root cause of the issue.

We'll likely want to change the command running the original container to point to directory that does exist upon container start. Alternatively, it may make sense to mount an ephemeral Volume to provide the directory.

```
vagrant@kubernetes-control-plane:~/lab$ vim pod-sol.yaml
vagrant@kubernetes-control-plane:~/lab$ cat pod-sol.yaml
apiVersion: v1
kind: Pod
metadata:
  name: date-recorder
spec:
  containers:
    - name: debian
      image: gcr.io/distroless/nodejs20-debian11
      command: ["/nodejs/bin/node", "-e", "const fs = require('fs'); let timestamp = Date.now(); fs.writeFile('/var/startup/startup-marker.txt', timestamp.toString(), err => { if (err) { console.error(err); } while(true) {} });"]
  volumeMounts:
    - mountPath: /var/startup
      name: init-volume
  volumes:
    - name: init-volume
      emptyDir: {}
vagrant@kubernetes-control-plane:~/lab$
```

```
vagrant@kubernetes-control-plane:~/lab$ kubectl delete -f pod.yaml
pod "date-recorder" deleted
vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f pod-sol.yaml
pod/date-recorder created
vagrant@kubernetes-control-plane:~/lab$ kubectl get pod
NAME             READY   STATUS    RESTARTS   AGE
date-recorder    1/1     Running   0           16s
vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$ kubectl logs date-recorder
vagrant@kubernetes-control-plane:~/lab$
```